Philotic Stack

Full Codebase Analysis

March 10, 2026

Prepared by Perplexity Computer

Language	Rust (Edition 2024 / 2021)
Repository	github.com/likesjx/philotic-stack
Workspace	8 crates, 69 source files
Lines of Code	~29,124
Tests	254 unit/integration tests
TODO/FIXME	1 (excellent discipline)

Table of Contents

Section 1

Architecture Analysis

1.1 Architecture Overview

The Philotic Stack is a distributed AI agent operating system built in Rust, inspired by the "Ansible" and "Philotic Web" metaphors from Orson Scott Card's Speaker for the Dead. The central metaphor: a Hotel is a node that materializes AI Guests (processes). All state lives in a SQLite Context Graph owned by the hotel daemon.

The architecture follows a Hub-and-Spoke (Hotel/Guest) pattern with inter-hotel mesh networking for distribution. It is NOT a flat microservices architecture — the Hotel is the single source of truth, and Guests are subordinate processes.

1.2 Crate Dependency Graph

Crate Dependency Flow

```
Dependency Flow (bottom-up):

  ansible-mesh-core     (Layer 0 - shared primitives, traits, types)
         |
  philotic-client       (Layer 1 - Guest SDK, IPC client only)
         |                     |
  ansible              agent-core
  (Hotel daemon)       hegemon
                       model-router
                       tool-runner
                       robot-kit  (standalone, optional)
```

Layer	Crate(s)	Purpose
Layer 0 (Foundation)	ansible-mesh-core	Shared primitives, traits, storage abstractions, beacon types
Layer 1 (SDK)	philotic-client	Guest IPC client library
Layer 2 (Runtime)	ansible	Hotel daemon — IPC, materialization, mesh, HTTP, auth
Layer 3 (Guests)	agent-core, hegemon, model-router, tool-runner	AI cognitive loop, gateway, LLM routing, tools
Layer 4 (Extension)	robot-kit	Standalone robotics HAL for Raspberry Pi

Table 1: Layered architecture of the Philotic Stack workspace.

1.3 Crate Size Table

Crate	Lines	Modules	Purpose	Maturity
ansible	9,984	10	Hotel daemon — IPC, materialization, mesh, HTTP, auth	Functional
agent-core	5,158	5	Agent cognitive loop, session management, approval workflow	Functional
ansible-mesh-core	4,586	18	Core primitives, traits, storage abstractions, beacon types	Functional
robot-kit	3,471	10	Robotics HAL for Raspberry Pi (drive, vision, speech, safety)	Scaffold+
model-router	2,273	7	LLM provider routing (Gemini, ElevenLabs)	Functional
philotic-client	1,505	1+ex.	Guest SDK — IPC client for hotel communication	Production-ready
hegemon	1,377	1	Telegram gateway, Markdown HTML rendering	Functional
tool-runner	770	1	Workspace file operations tool executor	Functional

Table 2: Crate sizes and maturity levels. Total: ~29,124 lines of Rust.

1.4 Communication Architecture

The system uses three distinct communication layers, each serving a different scope and use case:

1. Intra-Hotel (Unix Domain Sockets)

- Socket: `/tmp/philotic-ansible.sock`
- Protocol: Length-prefixed (4-byte big-endian u32) JSON frames
- Types: IpcRequest / IpcResponse enum variants
- Pattern: Request-response with interleaved push messages

2. Inter-Hotel (UDP Mesh)

- BeaconMessage envelopes on port 8999
- HMAC-PSK authentication (opt-in) with replay protection via nonce tracker (SQLite-backed)
- Message types: Heartbeat, MeshEventBatch, MeshEventAck, WebRtcSignal, ToolCall, etc.

3. Blob Store (HTTP)

- Content-addressed large payloads via Axum HTTP on port 9001

1.5 Data Flow — User Message Path

User Message Flow

```
User Message Flow:

  External msg (Telegram) --> hegemon (Gateway Guest)
      |
      +--> IPC --> Hotel (ansible) --> routes via inbox subscription
                       |
                  agent-core builds context
                       |
                  model-router --> LLM API (Gemini)
                       |
                  agent-core interprets (respond / tool_call / approval)
                       |
              +--------+--------+
              |                 |
        [tool_call]        [respond]
              |                 |
        tool-runner          IPC --> Hotel --> hegemon --> Telegram
              |
        result --> model (loop back)
```

1.6 Key Architectural Patterns

- Hotel as Single Source of Truth: All state in SQLite Context Graph
- Guest Lifecycle Management: Supervisor loop every 5s, auto-respawn dead guests
- Memory Apartments: LWW CRDT semantics for guest memory sync
- Role-Based Message Routing: Guests subscribe to inbox roles, Hotel routes by role
- Capability Advertisement: Heartbeat-based node registry with TTL-based freshness
- Durable Event Ledger: Append-only event log with per-node ACK cursors
- Pluggable Storage: Arc<dyn GraphStorage> trait abstraction over SQLite

1.7 Code Smell Assessment

Positive Indicators

- Extremely clean code with only 1 TODO marker across 29K lines
- 254 tests — excellent coverage for an early-stage project
- Consistent naming conventions (snake_case, descriptive names)
- Well-organized module structure with clear separation of concerns
- Strong use of Rust's type system (enums for message types, serde for serialization)
- Good documentation in README files for every crate
- AGENTS.md and CLAUDE.md show mature development process thinking
- Proper use of tracing for structured logging throughout

Concerns

- 117 unwrap() calls outside tests — should be 0 in production paths
- IpcResponse uses #[serde(untagged)] — fragile for forward evolution
- Large monolithic files: main.rs (2,892 lines), ipc.rs (4,911 lines), runtime.rs (2,251 lines), session.rs (2,042 lines)
- anyhow used everywhere — no domain-specific error types (thiserror only in robot-kit)
- PhiloticClient holds a single UnixStream — no connection pooling or reconnection
- Cutover flags suggest transitional code that may calcify
- robot-kit has a separate edition (2021) and feels bolted on vs. integrated

1.8 Current vs. Target Architecture

Current State	Target State	Gap
Single-node hotel with local guest materialization	Multi-hotel mesh networking with durable event delivery	Mesh dispatcher partially implemented; gated behind feature flags
IPC communication fully functional	Full WebRTC data channels for media streaming	WebRTC is signaling-only scaffold
Gemini model routing working	Model Manager as addressable mesh unit for intelligent routing	Types exist but no routing intelligence
Session management with turns, approval, tools	Fine-grained policy engine	Current approval workflow is basic
Memory apartment sync (LWW)	Context Graph as distributed CRDT state	No CRDT beyond LWW apartments
Telegram integration via hegemon	Multiple transport gateways + iOS beacon	iOS beacon not started
Hardcoded tool definitions	MCP integration for tool execution	No MCP protocol implementation yet
Isolated agent sessions	Multi-agent collaboration / delegation	No agent-to-agent delegation

Table 3: Gap analysis between current implementation and design goals.

Section 2

Component Analysis

2.1 philotic-client

Layer 0 — Guest SDK | Lines: 1,505 | Tests: 3 | Maturity: Production-ready

Purpose

The universal IPC client SDK that all Guest processes use to communicate with their Hotel. Defines the IpcRequest/IpcResponse protocol and provides async connection + message framing.

Code Quality

Excellent. Clean, focused API. Good error handling with anyhow contexts. Tests cover connection, interleaved push messages, and disconnect detection. The frame protocol (4-byte BE length prefix) is simple and correct.

Issues

- Single UnixStream with no reconnection logic — if the hotel restarts, guests crash
- IpcResponse uses #[serde(untagged)] — adding new response variants is fragile
- No backpressure mechanism for push messages (VecDeque grows unbounded)
- No timeout on read_frame — can hang indefinitely

Opportunities

- Add automatic reconnection with exponential backoff
- Switch IpcResponse to #[serde(tag = "type")] for safer evolution
- Add a bounded channel for push messages with backpressure
- Add heartbeat/keepalive to detect dead connections proactively
- Consider splitting into philotic-types (shared types) + philotic-client (runtime)

2.2 ansible-mesh-core

Layer 0 — Foundation | Lines: 4,586 | Tests: 47+ | Maturity: Functional

Purpose

The shared foundation crate. Defines all core types (BeaconMessage, NodeCapabilities, EventEnvelope), storage traits (GraphStorage, EventStorage, CursorStorage, GraphAdapter), and mesh primitives (beacon, heartbeat, registry, authz).

Code Quality

Well-structured with 18 clearly-separated modules. Storage traits are well-designed with clear contracts. The SQLite implementation (1,302 lines) is thorough with proper migrations. Parity harness tests ensure type compatibility.

Issues

- Type aliases (NodeId = String, AgentId = String) provide no type safety — interchangeable by accident
- MeshAuth uses a single PSK for all nodes — no per-node key rotation
- NonceTracker stores in SQLite — performance concern at high message rates
- Many modules are thin (materializer: 23 lines, tools: 33 lines) suggesting premature API surface
- GraphAdapter trait defined but has no SQLite implementation

Opportunities

- Use newtypes (struct NodeId(String)) instead of type aliases for compile-time safety
- Implement per-node key derivation from a master PSK
- Consider in-memory bloom filter for nonce tracking with periodic SQLite flush
- Consolidate thin modules or mark them as WIP

2.3 ansible

Layer 2 — Hotel Daemon | Lines: 9,984 | Tests: 45+ | Maturity: Functional

Purpose

The Hotel daemon — the central orchestrator. Manages guest materialization, IPC server, mesh beacon, blob storage, vault, and HTTP API. This is the hub of the entire system.

Code Quality

Most complex crate. main.rs at 2,892 lines and service/ipc.rs at 4,911 lines are too large. However, the code within is well-organized with clear responsibility boundaries. The GuestManager with Materializer trait is a clean abstraction.

Issues

- main.rs is a God file — startup, config, test harness, Axum routes, fake Telegram server all in one
- service/ipc.rs at 4,911 lines handles ALL IPC operations
- Cutover flags (AnsibleCutoverFlags) suggest an incomplete migration
- FakeTelegramState in main.rs is test infrastructure mixed with production code
- guest_manager.rs opens a "throwaway" SQLite connection as a backwards-compatibility hack

Opportunities

- Break main.rs into: startup.rs, config.rs, routes.rs, test_harness.rs
- Break ipc.rs into: ipc_server.rs, ipc_routing.rs, ipc_session.rs, ipc_tools.rs
- Move test infrastructure to a test_support module
- Implement proper graceful shutdown
- Add health check endpoint to the HTTP API

2.4 agent-core

Layer 3 — Cognitive Loop | Lines: 5,158 | Tests: 53+ | Maturity: Functional

Purpose

The agent personality/cognitive loop. Manages sessions, interprets model responses, handles tool calls, routes approval requests, and maintains conversational context. This is "Jane's brain."

Code Quality

Well-designed agent loop with clear phase transitions (TurnPhase enum). Sophisticated session management with tool assembly, component routing, approval policies, and workspace bindings. Extensive tests.

Issues

- runtime.rs at 2,251 lines is too large — handles user messages, model responses, tool results, approval commands, session loading, memory sync, all in one struct
- session.rs at 2,042 lines has massive build_prompt_with_tools and checkpoint_json methods
- "TOOL:echo" prefix parsing is fragile text-based tool invocation that should be structured
- interpret_tool_result always wraps as "Tool X says: Y" — single-turn tool use only
- Recent turns capped at 8 with no sliding window or summarization
- No retry logic for failed model calls; no token counting or context window management

Opportunities

- Extract runtime.rs into: message_handler.rs, model_handler.rs, tool_handler.rs, approval_handler.rs
- Implement multi-turn tool use (tool result feeds back to model, not auto-responded)
- Add token counting to prevent context window overflow
- Implement conversation summarization beyond 8 turns
- Add retry with exponential backoff for transient model failures

---

2.5 hegemon

Layer 3 — Telegram Gateway | Lines: 1,377 | Tests: 10 | Maturity: Functional

Purpose

Telegram Bot API gateway. Polls for updates, translates Telegram messages to IPC tasks, renders Markdown responses as Telegram-compatible HTML, handles photos/voice/documents via blob service.

Code Quality

TelegramHtmlRenderer is well-implemented and thoroughly tested. Clean separation between Telegram protocol handling and IPC communication. Good handling of attachment types.

Issues

- Entire crate is a single main.rs file (1,377 lines)
- Polling-based with hardcoded timeout — not webhook-based
- Telegram API key stored as config value, not vault secret
- No rate limiting for Telegram API calls

Opportunities

- Split into: telegram_client.rs, html_renderer.rs, attachment_handler.rs
- Support webhook mode for production (more efficient than polling)

- Use vault for API key storage
- Add message chunking for responses exceeding Telegram's 4096-char limit
- Generalize as a "transport gateway" pattern for Discord, Slack, etc.

2.6 model-router

Layer 3 — LLM Routing | Lines: 2,273 | Tests: 22+ | Maturity: Functional

Purpose

Routes model inference requests to LLM providers. Currently supports Gemini (text, vision, voice) and ElevenLabs (TTS). Uses a ModelController trait for provider abstraction.

Code Quality

Clean trait abstraction (ModelController). Good handling of Gemini's multimodal capabilities including vision and voice. The ContextEnvelope system for prompt construction is well-designed with projection items.

Issues

- Only 2 providers implemented (Gemini, ElevenLabs) — no OpenAI, Anthropic, or local models
- No model selection logic — tasks are statically routed by role name
- No fallback/retry across providers
- No cost tracking or token metering
- Voice synthesis is Gemini-specific, not abstracted

Opportunities

- Add OpenAI and Anthropic providers
- Implement intelligent model selection (cost, latency, capability matching)
- Add fallback chains (Gemini OpenAI local)
- Implement token metering and cost tracking per session
- Add streaming response support (currently buffered)

2.7 tool-runner

Layer 3 — Workspace Tool Executor | Lines: 770 | Tests: 11 | Maturity: Functional

Purpose

Executes workspace file operations (list, read, search) within a sandboxed workspace reference. Applies per-request overlays from the agent for dynamic configuration.

Code Quality

Clean implementation with proper path sanitization. Good use of overlay pattern for per-request configuration. Sensible defaults with env var overrides.

Issues

- Only 3 tools implemented (workspace.list, workspace.read, workspace.search)
- No tool discovery/registration protocol — tools are hardcoded

- No execution timeout
- Path sanitization could be more robust (no symlink resolution)

Opportunities

- Implement MCP protocol for tool discovery and execution
- Add more tools (workspace.write, workspace.execute, http.get, etc.)
- Add execution timeouts and resource limits
- Implement tool sandboxing beyond path checks

2.8 robot-kit

Layer 4 — Extension | Lines: 3,471 | Tests: 55+ | Maturity: Scaffold+

Purpose

Robotics HAL for Raspberry Pi integration. Provides drive, vision (Ollama), speech, sensors, safety, and emote modules. Designed to be compatible with ZeroClaw's Tool trait.

Code Quality

Well-documented with feature flags and conditional compilation for Pi-specific GPIO. The Tool trait is clean and reusable. Comprehensive test suite including safety checks.

Issues

- Completely disconnected from the hotel/guest architecture — no IPC integration
- Uses its own ToolResult type, separate from agent-core's ToolResult
- Config loaded from TOML files, not from Context Graph
- No actual hardware testing possible without a Pi
- Edition 2021 vs. 2024 for the rest of the workspace

Opportunities

- Bridge to hotel via PhiloticClient — each robot tool becomes an IPC-connected guest
- Unify ToolResult types across crates
- Implement as an MCP server for seamless agent integration
- Add a simulation mode for development without hardware

Section 3

Agent Framework Assessment

3.1 Execution Model

The Philotic Stack implements a message-passing, event-driven execution model with distinct phases. This is a single-threaded, sequential agent loop — one turn at a time per session. The Hotel multiplexes across multiple concurrent sessions via role-based inbox routing.

Agent Turn Phase Transitions

```
TurnPhase State Machine:

  Queued --> LoadingContext --> WaitingModel --> Thinking
                                                   |
                              +--------------------+--------------------+
                              |                    |                    |
                        WaitingTool          WaitingApproval       Completed
                              |                    |
                              +----> Thinking <----+
                                        |
                                   Completed / Failed
```

3.2 How It Handles Key Concerns

Concern	Implementation
Agent Lifecycle	GuestManager + Materializer trait. OS process supervision with 5s reconciliation loop. Ghost reclamation on restart. PID tracking in Context Graph.
Memory	Memory apartments (LWW CRDT) in SQLite. Per-agent, per-type memory slots. Agent profile (persona, soul, identity) loaded from Context Graph. Recent turns buffer (8 max). No long-term summarization.
Tool Use	ToolDefinition registered per session. Structured agent_action for tool calls. Tool execution routed via IPC to specialized tool-runner guests. Approval workflow for sensitive tools.
Message Routing	Role-based inbox subscriptions. Hotel dispatches by target_role. Guests register with supported_tools list. Node registry tracks cross-hotel capabilities.
Context Mgmt	SessionBindings configure per-session tool sets, workspace refs, component routes. ContextEnvelope in model-router organizes prompt sections (instructions, identity, memory, tools, turns).

Table 4: How the Philotic Stack addresses core agent framework concerns.

3.3 What's Missing vs. Production Agent Frameworks

1. Multi-turn tool use: Current implementation auto-responds after one tool call. Production frameworks support iterative tool model tool loops.
2. Streaming: All responses are buffered. No SSE/streaming support from model to gateway.
3. Structured output: No JSON schema enforcement on model responses. Agent_action parsing relies on model compliance.
4. Planning/reasoning: No ReAct, Plan-and-Execute, or Tree-of-Thought. Pure single-step respond-or-call-tool.
5. Token management: No counting, no context window awareness, no automatic summarization.
6. Observability: tracing crate used but no OpenTelemetry, no structured spans, no distributed tracing.
7. MCP integration: No Model Context Protocol implementation despite being a design goal.
8. Multi-agent collaboration: No agent-to-agent delegation. Each agent is isolated in its session.
9. Error recovery: No retry logic, no circuit breakers, no fallback strategies.
10. Integration testing: No tests exercising full Hotel Guest Model Tool roundtrip.

3.4 Framework Design Strengths

1. Clean separation of concerns: The Hotel/Guest boundary is well-defined and enforced by IPC.
2. Crash safety: Guest supervisor auto-restores failed processes — production-critical.
3. Pluggable storage: Arc<dyn GraphStorage> means you can swap SQLite for any backend.
4. Security foundations: HMAC mesh auth, vault for secrets, approval workflow, path sanitization.
5. The metaphor works: Hotel/Guest/Apartment/Materializer creates a consistent, understandable mental model.
6. Process isolation: Each guest is a separate OS process — fault isolation by default.
7. Extensible protocol: The IpcRequest/IpcResponse enum is easy to extend with new operations.

Section 4

Framework Comparison & Recommendations

4.1 Framework Landscape Overview

The agentic AI framework landscape as of early 2026 includes over a dozen major frameworks across Rust, Python, TypeScript, and multi-language platforms. The following analysis draws on comprehensive research into each framework's architecture, strengths, and production readiness to position the Philotic Stack within the broader ecosystem.

- Rig (Rust): Modular crate-based library with compile-time capability enforcement. 20+ model providers, 10+ vector stores, Tool RAGging, WASM compatible. The only other Rust-native agent framework. v0.31 (Feb 2026).
- LangGraph (Python): Directed graph (Pregel-inspired) model with reducer-driven state, persistent checkpointing, and HITL. Explicit, auditable control flow. Largest Python agent ecosystem. Stable 1.0 release (2025).
- CrewAI (Python): Role-based crews with Flows API for deterministic orchestration. 44K+ stars, 12M+ monthly downloads. Native MCP + A2A support. 1.7 billion workflows processed.
- Google ADK (Python/Java/Go): Hierarchical multi-agent with Workflow Agents + LLM-driven routing. Native A2A protocol. 100+ Google Cloud connectors. Bidirectional audio/video streaming.
- OpenAI Agents SDK (Python/TS): Five primitives: Agent, Runner, Handoffs, Guardrails, Tracing. Fastest time to working agent. Deep OpenAI ecosystem integration. MCP server support.
- Letta/MemGPT (Python): Best-in-class persistent memory with core/archival tiers. Sleep-time compute for background memory consolidation. Context Repositories (git-based memory). Model-agnostic.
- Anthropic Patterns (Any): Five workflow patterns: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer. Computer Use tool. Conservative, well-tested at scale.
- Microsoft Agent Framework (Python/.NET): Unified successor to Semantic Kernel + AutoGen. Graph-based workflows with streaming and checkpointing. A2A, AG-UI, and MCP standards. Release Candidate (Feb 2026).

4.2 Comparison Matrix

Framework	Lang.	Architecture	Memory	MCP	A2A	Maturity
Rig	Rust	Modular crates	Vector stores	Yes	No	Mod-High
LangGraph	Python	Directed graph	Checkpointing	Yes	Yes	High
CrewAI	Python	Role-based crews	STM/LTM/Entity	Yes	Yes	High

Framework	Lang.	Architecture	Memory	MCP	A2A	Maturity
Google ADK	Py/Java/Go	Hierarchical	Session-based	Yes	Yes	High
OpenAI SDK	Python/TS	Loop + handoffs	External	Yes	No	Mod-High
Letta	Python	Stateful memory	Core/archival	No	No	High
Anthropic	Any	API patterns	External	Via API	N/A	High
MS Agent Fw	Py/.NET	Graph workflows	External	Yes	Yes	High
Philotic Stack	Rust	Hub-spoke mesh	LWW/SQLite	TBD	TBD	Early

Table 5: Framework comparison matrix. Philotic Stack is the only Rust-native distributed agent framework.

4.3 Philotic Stack's Unique Position

The Philotic Stack occupies a distinct niche in the agent framework landscape. No other framework combines all of these characteristics:

- Only Rust-native distributed agent framework — Rig is Rust but not distributed; all others are Python/TypeScript
- Only framework with physical mesh networking (WireGuard/UDP) vs. HTTP-only communication
- Only framework with OS-level process isolation for agents (vs. in-process threads)
- Hotel/Guest metaphor provides clearer boundaries than actor model alone — a consistent, understandable mental model that maps directly to infrastructure concerns

4.4 Ideas to Adopt from Other Frameworks

Source	Key Ideas
Rig	Compile-time capability enforcement via Rust type system; Tool RAGging (store tool definitions in vector store, retrieve at runtime)
LangGraph	Explicit graph-based workflow definition; checkpoint-based persistence for long-running workflows
CrewAI	Role-based agent definitions with Flows API for deterministic orchestration backbone
Letta/MemGPT	Sleep-time compute for background memory consolidation; Context Repositories (git-based memory storage)
Google ADK	A2A protocol for cross-framework interoperability; bidirectional streaming

Source	Key Ideas
Anthropic	The 5 workflow patterns: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer

Table 6: Key ideas to adopt from leading agent frameworks.

4.5 Critical Ideas to Explore

1. MCP as universal tool interface: Replace custom tool definitions with MCP servers for standardized tool discovery and execution.

2. A2A protocol for inter-hotel communication: Complements the UDP mesh with a standard agent-to-agent interoperability protocol.

3. Sleep-time compute: Background memory consolidation during agent downtime, inspired by Letta/MemGPT.

4. Cost-aware model routing: Router-R1 and MasRouter patterns from NeurIPS/ACL 2025 for intelligent model selection.

5. Confidence-aware routing: Use model self-assessment to decide when to escalate to more capable (expensive) models.

6. Context Repositories: Git-based memory storage for version-controlled agent knowledge, enabling branching and rollback.

7. CodeAct pattern: Agents that write and execute code rather than just calling predefined tools, dramatically expanding capability surface.

Section 5

Prioritized Recommendations

Recommendations are organized by urgency and impact. "Must Address" items are blocking issues that affect reliability and correctness. "Should Address" items are high-value improvements. "Explore" items are strategic investments for the next phase of development.

5.1 Must Address (Blocking)

These items represent risks to system reliability and developer productivity that should be resolved before expanding the feature surface.

1. Eliminate unwrap() calls in production paths — 117 found outside tests. Each is a potential panic. Convert to proper error handling with ? operator and anyhow/thiserror.
2. Add reconnection logic to PhiloticClient — A single UnixStream with no reconnect means hotel restarts kill all guests permanently. Implement exponential backoff reconnection.
3. Break up God files — ipc.rs (4,911 lines), main.rs (2,892 lines), runtime.rs (2,251 lines), session.rs (2,042 lines) hinder maintainability. Extract into focused modules.
4. Implement multi-turn tool use — Current single-turn auto-response prevents iterative reasoning. Feed tool results back to the model for follow-up decisions.
5. Add token counting and context window management — Without this, sessions exceeding the context window will silently truncate or fail. Critical for production reliability.

5.2 Should Address (High Value)

6. Implement MCP protocol for standardized tool integration and discovery.
7. Add streaming response support end-to-end from model to gateway.
8. Switch IpcResponse to tagged enum (#[serde(tag = "type")]) for protocol safety and forward evolution.
9. Add OpenTelemetry for distributed tracing across Hotel Guest boundaries.
10. Implement at least one more model provider (OpenAI or Anthropic) to validate ModelController abstraction.
11. Add integration tests for full message roundtrip (Hotel Guest Model Tool Response).
12. Implement conversation summarization beyond the 8-turn buffer for long-running sessions.

5.3 Explore (Strategic)

13. A2A protocol for inter-hotel interoperability with other agent frameworks.
14. Sleep-time compute for background memory consolidation during agent downtime.
15. Graph-based workflow definitions (LangGraph-inspired) for complex multi-step agent workflows.
16. iOS beacon prototype (Swift + Rust UniFFI) for mobile mesh participation.

17. CRDT expansion beyond LWW — OR-Set for capabilities, G-Counter for metrics, enabling true distributed state.

This analysis was prepared on March 10, 2026. Framework comparison data reflects the state of the agentic AI ecosystem as of early 2026. The Philotic Stack is at an early but promising stage with strong architectural foundations. The recommendations above provide a roadmap from functional prototype to production-grade distributed agent operating system.

Sources: Framework research based on official documentation and repositories including Rig, LangGraph, CrewAI, Google ADK, OpenAI Agents SDK, Letta/MemGPT, Anthropic Agent Patterns, and Microsoft Agent Framework. Codebase data from github.com/likesjx/philotic-stack.